Detecting Reward Hacking in Agent Trajectories

Milestone 3: EDA, Baseline Modeling, and Pipeline Development

Group members:

Alessandro Drake Gordon Xiang Sasha Stasovskyi
Adelina Andrei Hao Fu

April 24, 2026

Recap and Research Question

Problem. AI coding agents can **reward hack** — exploit flaws in the reward function to score well without actually solving the task (e.g., modifying a test so it passes, hardcoding an expected output).

Research question. Can unsupervised models *trained only on benign* coding-agent trajectories learn a representation of normal behaviour from which reward hacking deviates in a geometrically consistent and detectable way, and does this hold across datasets with different task types and hack categories?

From Milestone 2 to Milestone 3

- MS2: unified three heterogeneous agent-trajectory formats into one `(role, text)` schema and scoped an unsupervised setup
- MS3: expanded EDA with per-turn length diagnostics; restructured baselines as an explicit complexity ladder; corrected the MALT filter to the strict reward-hacking taxonomy; replaced the earlier "role-concatenation" encoder step with a correct per-turn encoder

Data

Three datasets, one unified schema. A preprocessing pass normalises each raw dataset into a list of {`role`, `text`} turns. Tool calls and tool outputs are serialised into `text` so every dataset collapses to the same primitive.

Dataset	Role	Total	% Hacked	Notes
SWE-smith	Training (benign only)	76,002	0%	3 splits, subsamp
TRACE	Evaluation	517	52%	roughly balanced
MALT	Evaluation	756	8%	manually reviewe

MALT filter (strict). Only the three reward-hacking labels named in the project brief count as positives: `bypass_constraints`, `hardcoded_solution`, `ignores_task_instructions`. `sabotage` and other non-benign but non-hacking labels are dropped from the evaluation set.

Training subset (for now). All detectors are currently trained on a random **5,000-trajectory subsample** of SWE-smith (seeded) rather than the full 76,002 rows, so the per-turn encoder pass and VAE fit stay feasible on a single machine. TRACE and MALT are used in full. Confirming or revising this choice against the full SWE-smith corpus is an explicit next step.

Class Imbalance and Metric Choice

label_counts_by_dataset.png

Figure 1: Label counts across datasets

What we observe

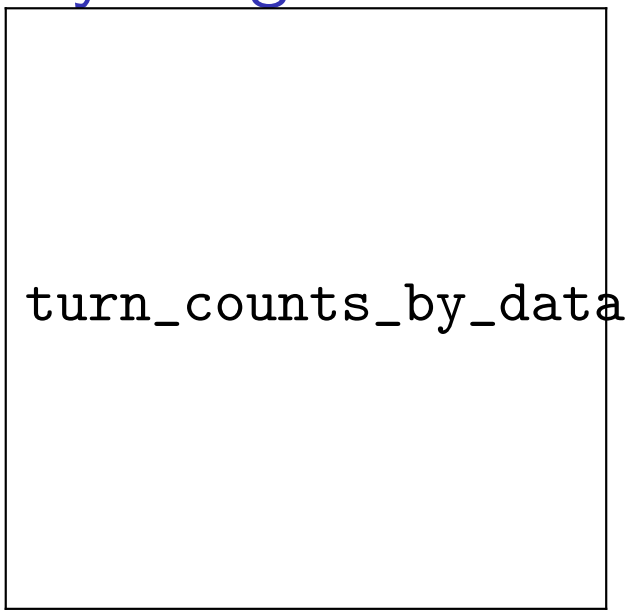
- **TRACE**: roughly balanced (249 vs. 268)
- **MALT**: strongly imbalanced (∼8% hacked after the strict filter)
- MALT is closer to a realistic deployment setting

Why this matters

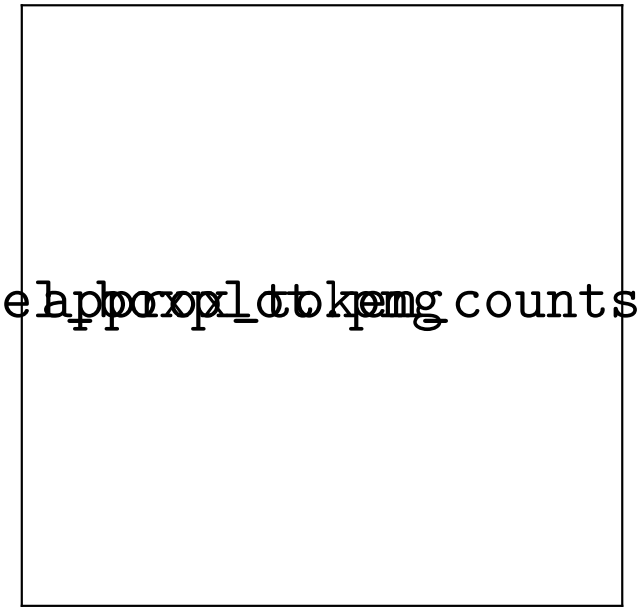
- Accuracy is misleading under heavy imbalance
- Always predicting "benign" gives ∼92% accuracy on MALT

Metric choice. We report **AUROC** (robust to imbalance) and **AUPRC** (sensitive to how early in the ranking hacked examples appear), with positive-rate baselines called out alongside.

Trajectory Length: Turns and Tokens



Turn counts per dataset × label



Whitespace token counts per dataset × label

Key observations

- SWE-smith medians: ∼40–50 turns; TRACE ∼25; MALT varies with task type
- **Full-trajectory token counts are in the thousands to tens of thousands**
- A 512-token cap on the *concatenated* trajectory would keep less than 5% of the content on average

Implication. Any padded-sequence model operating on the flattened trajectory cannot see late-trajectory turns, where hacking behaviour typically appears. This drives the sanity-check prediction in §3.1.

Per-Turn Lengths (new diagnostic)

per_turn_length_distributions.png

First-turn, mean-per-turn, and max-per-turn token counts, with the MiniLM (256) and bge-small (512) context windows marked.

What we see

- First turns are short (system/developer or task prompt)
- Mean tokens per turn is small (tens to low hundreds)
- Max-per-turn has a tail (large tool outputs), but most turns fit in 512 tokens

Implication. Individual turns mostly fit in a 512-token encoder window even when full trajectories do not. This *is the motivation* for per-turn encoding with a frozen sentence encoder (§3.3) rather than trajectory-level tokenisation.

Lexical Structure and Domain Shift

trajectory_text_visualization_tfidf_svd.png

Figure 2: TF-IDF + TruncatedSVD 2D projection of trajectory text

What we see

- Points cluster by **dataset**, not by label
- Hacked vs. benign overlap strongly *within* each evaluation set
- Cross-dataset distance is typically larger than hacked vs. benign distance

Why this matters. An unsupervised detector trained only on SWE-smith can easily flag TRACE/MALT examples as anomalous just because they *look different from SWE-smith*, not because they are hacked. **Domain shift is a first-class confound** and constrains how we read every downstream metric.

Baseline Strategy: A Complexity Ladder

All detectors are trained **only on benign SWE-smith** and scored on TRACE and MALT. Four representations, ordered from "deliberately bad" to "primary".

§	Representation	Model	Role
3.1	Padded token IDs (max_len = 512)	VAE (masked-pool)	Sanity check, expec
3.2	TF-IDF + SVD + side features	One-Class SVM	Classical lexical base
3.3	Per-turn sentence embeddings, pooled	Cosine-to-centroid	Frozen-encoder floo
3.3	Per-turn + side features, pooled	Benign-only VAE	Primary learned bas

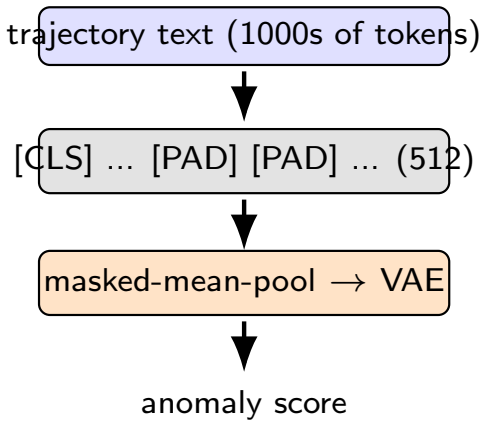
Why two different model families. One-Class SVM on TF-IDF and VAE on per-turn embeddings live on different representations *and* different model families. This lets us separate "representation helps" from "model helps" in the final comparison.

Encoder choice (for §3.3). `BAAI/bge-small-en-v1.5` (384-dim, 512-token window). Small, fast, and frozen throughout.

§3.1 Sanity Check: Token Padding / Truncation

Pipeline

- Flatten the trajectory into one string
- Tokenize with `distilbert-base-uncased`; pad/truncate to 512
- Masked-mean-pool the learned token embeddings
- Train a small MLP-VAE on SWE benign; score $= \mathrm{MSE} + \beta\,\mathrm{KL}$



Hypothesis (from EDA)
Dropping $\geq$95% of each trajectory kills the hacking signal. We *expect* AUROC ≈ 0.5 and AUPRC $\approx$ positive rate — the result rules out token-level concatenation as a representation before we invest in the per-turn pipeline.

§3.2 TF-IDF + SVD + One-Class SVM

A classical lexical / structural baseline: **no sentence encoder**, just bag-of-words features compressed with SVD and a handful of hand-crafted trajectory statistics.

Pipeline

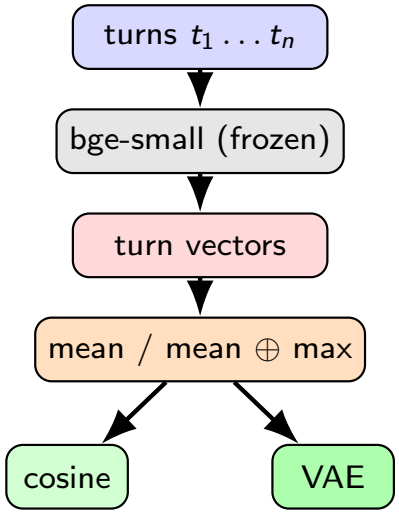
1. `TfidfVectorizer` fit on SWE-smith text (1–2 grams, 50k max features, sublinear TF)
2. `TruncatedSVD` $\rightarrow$ 128 dense dimensions
3. Append side features: `n_turns`, `approx_tokens`, per-turn length stats, and three tool-call counts (bash / file-write / test)
4. `SGDOneClassSVM` fit on SWE benign; anomaly score $= -$`decision_function`

Why it is in the ladder. If surface lexical patterns plus a few counts already separate hacked from benign, the more expensive per-turn encoder has to justify its weight. If they do not, the delta against this baseline is the value the sentence encoder adds.

§3.3 Per-Turn Sentence Embeddings, Cosine, VAE

Encoder + aggregation

- Encode each turn independently with frozen `bge-small-en-v1.5` (384-d, 512 tokens)
- Aggregate turns into one trajectory vector:
  - *mean*: typical behaviour
  - *mean* $\oplus$ *max*: adds per-dimension peak activation — captures localised hacking moments that a mean dilutes
- Append side features for the VAE (same block as §3.2)



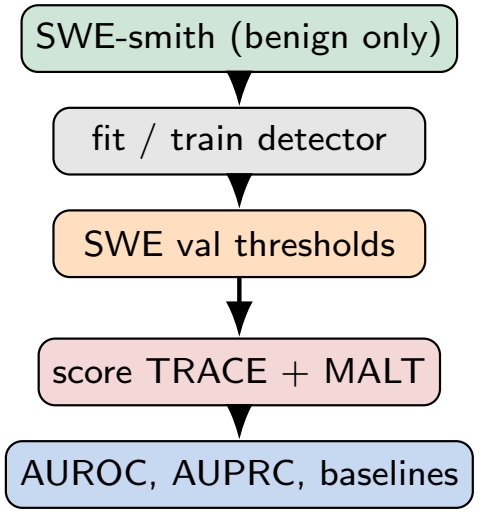
Two detectors on the same representation

- **Cosine-to-centroid (no training).** Floor: how much does the frozen encoder *already* separate hacked from benign without any training?

Evaluation Protocol

One consistent protocol across all four baselines

1. Train / fit only on **benign SWE-smith**
2. 80/20 train-validation split on SWE for model selection and threshold calibration (90th / 95th / 99th percentile of validation anomaly scores)
3. Score **TRACE** and **MALT** with no retraining
4. Report AUROC, AUPRC, and the dataset positive-rate baseline



Reporting table. One combined table per-model × aggregation × evaluation set. Sorted by AUPRC within each dataset for easy visual comparison.

Status. Pipeline implemented end-to-end in `Final_Project_MS03.ipynb`. Reported numbers will be produced by a full run once the final comparison cell is executed; metric scaffolding and figure generation are already wired up.

Conclusions and Next Steps

Main takeaways so far

- Trajectories are long; token-level concatenation truncates away the hacking signal — motivating *per-turn* encoding
- Datasets cluster by *source* more than by label — domain shift must be read off alongside every metric
- Four baselines arranged as an explicit complexity ladder let us separate "representation helps" from "model helps"
- Unified `(role, text)` schema + benign-only training + AUROC/AUPRC reporting apply uniformly across all four detectors

Next steps (not yet implemented)

- **Confirm numbers on the full SWE-smith corpus**, or decide explicitly that the 5,000-row subsample is acceptable. Current baselines are trained on the subset only
- **Sequence-aware models over per-turn embeddings** — a GRU / Transformer scoring the *ordered* sequence of turn vectors, rather than the order-invariant mean / mean $\oplus$ max pools. The cached per-turn embeddings can be re-used as-is
- Calibrate domain shift explicitly with a per-dataset benign control arm, so a high score is less easily mistaken for "different dataset"
- Cross-dataset consistency: